

PL/SQL Exercise 1

Scenario 1 : The bank wants to apply a discount to loan interest rates for customers above 60 years old.

Qn : Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates

Code :

```
create table customers (customer_id number primary key, customer_name
varchar2(50), age number);
create table loans (loan_id number primary key, customer_id number,
interest_rate number);
```

```
insert into customers values (1, 'ram', 65);
insert into customers values (2, 'shyam', 45);
insert into customers values (3, 'rita', 70);
```

```
insert into loans values (101, 1, 10);
insert into loans values (102, 2, 12);
insert into loans values (103, 3, 11);
```

```
commit;
```

```
declare
    cursor c_customers is
        select customer_id, age from customers;

begin
    for cust in c_customers loop
        if cust.age > 60 then
            update loans set interest_rate = interest_rate - 1 where customer_id =
cust.customer_id;
        end if;
    end loop;
    commit;
    dbms_output.put_line('1% discount applied to eligible customers.');
```

```
end;
/
```

Output :

1% discount applied to eligible customers .
PL/SQL procedure successfully completed.

Scenario 2 : A customer can be promoted to VIP status based on their balance.

Qn : Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Code :

```
create table customers_vip (customer_id number primary key, customer_name varchar2(50), balance number, isvip varchar2(5));

insert into customers_vip values (1, 'ram', 15000, 'false');
insert into customers_vip values (2, 'shyam', 8000, 'false');
insert into customers_vip values (3, 'rita', 25000, 'false');
```

```
commit;
```

```
declare
    cursor c_customers is
        select customer_id, balance from customers_vip;
begin
    for cust in c_customers loop
        if cust.balance > 10000 then
            update customers_vip set isvip = 'true' where customer_id = cust.customer_id;
        end if;
    end loop;
```

```
    commit;
    dbms_output.put_line('VIP status updated.');
```

```
end;
/
```

Output :

VIP status updated .

PL/SQL procedure successfully completed .

Scenario 3 : The bank wants to send reminders to customers whose loans are due within the next 30 days.

Qn : Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Code :

```
create table loan_reminders (loan_id number primary key, customer_id number, due_date date);

insert into loan_reminders values (201, 1, sysdate + 10);
insert into loan_reminders values (202, 2, sysdate + 20);
insert into loan_reminders values (203, 3, sysdate + 40);
commit;

declare
    cursor c_customers is
        select customer_id, loan_id, due_date from loan_reminders where due_date between sysdate and
sysdate + 30;
begin
    for cust in c_customers loop
        dbms_output.put_line('Reminder: Loan ' || cust.loan_id || ' for Customer ' || cust.customer_id
|| ' is due on ' || to_char(cust.due_date, 'DD-MON-YYYY')
        );
    end loop;
end;
/
```

Output :

Reminder: Loan 201 for Customer 1 is due on 01-JUL-2026

Reminder: Loan 202 for Customer 2 is due on 11-JUL-2026

PL/SQL procedure successfully completed.